



• MODULE

DEPLOYING ML MODELS WITH STREAMLIT

Take your model from a notebook on your laptop to a public URL
anyone in the world can use.



WHAT WE'LL COVER

01

Save Your Model

Write the trained model to disk so it can travel.

02

Build the App

Wrap the model in a tiny Streamlit script.

03

Project Structure

Three files in one folder. That is the whole project.

04

Push to GitHub

Get your code onto a place the cloud can read it from.

05

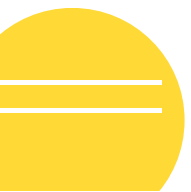
Deploy to the Cloud

Free hosting on Streamlit Community Cloud, in two minutes.

06

Common Pitfalls

The three errors that catch almost everyone the first time.





• THE MOTIVATION

A MODEL ON YOUR LAPTOP HELPS NOBODY

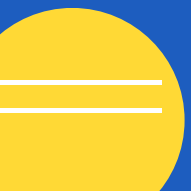
When you finish training a model in a notebook, only one person in the world can actually use it: you. To share it with a friend, a lecturer, an employer, or even just to test it on your own phone, the model has to live somewhere on the internet, with a friendly button people can press. Deployment is how we cross that gap.

- You can put the live URL on your CV and LinkedIn.
 - Non-technical people can try it without installing Python.
 - You can collect real feedback on how the model behaves.
- 

- THE TOOL

WHAT IS STREAMLIT?

Streamlit is a Python library that turns a regular `.py` script into a web app. You write `st.title("Hello")` and a heading appears in the browser. You write `st.number_input("Age")` and a number box appears. There is no HTML to learn, no JavaScript, no front-end framework. You write Python the same way you do in a notebook, and Streamlit handles the web part for you.



• THE JOURNEY

FROM NOTEBOOK TO PUBLIC URL

Six steps, in order. Each one is small. Together they take a model out of your notebook and put it in front of anyone with a browser.

01

Train

Fit a model in your notebook.

02

Save

Write it to `model.pkl`.

03

Wrap

Put it inside `app.py`.

04

Test

Run it locally and click around.

05

Push

Send the folder to GitHub.

06

Deploy

Point Streamlit Cloud at the repo.



• STEP 1

SAVE YOUR MODEL

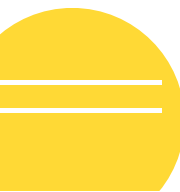
After training, your model is a Python object sitting in memory. The moment you close the notebook, it disappears. We use `joblib` to write it to disk as a single file we can carry around, the same way you would save a photo.

```
import joblib
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)

# Write the trained model to a single file
joblib.dump(model, "model.pkl")
# model.pkl is now sitting next to your notebook
```

NOTEBOOK



• STEP 2

WRAP THE MODEL IN A TINY APP

Create a new file called `app.py` in the same folder as `model.pkl`. The pattern is always the same: load the model once, ask the user for inputs, predict, show the result.

```
APP.PY

import joblib
import streamlit as st

model = joblib.load("model.pkl")

st.title("Iris Species Predictor")

sepal_length = st.number_input("Sepal length (cm)", value=5.0)
sepal_width = st.number_input("Sepal width (cm)", value=3.0)
petal_length = st.number_input("Petal length (cm)", value=1.5)
petal_width = st.number_input("Petal width (cm)", value=0.2)

if st.button("Predict"):
    prediction = model.predict([[sepal_length, sepal_width, petal_length, petal_width]])
    st.success(f"Predicted species: {prediction[0]}")
```



• STEP 3

RUN IT LOCALLY FIRST

Open a terminal in the project folder and run the command below. A new browser tab opens at `localhost:8501` showing your app. Get everything working here, with real inputs and real predictions, before you think about deployment.

TERMINAL

```
$ streamlit run app.py
```

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501

! If it does not work on your laptop, it will not work on the cloud either. Local first, cloud second.

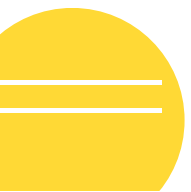


• STEP 4

THREE FILES IN ONE FOLDER

That is the entire project. No subfolders, no config files, no Dockerfile. Streamlit Cloud is happiest when everything sits at the top level.

```
my-streamlit-app/ ├── app.py ├── model.pkl └──  
requirements.txt
```

- **app.py**, the Streamlit script we just wrote.
 - **model.pkl**, the saved model. Must be in the same folder.
 - **requirements.txt**, the list of Python libraries the cloud needs to install.
- 

TELL THE CLOUD WHAT TO INSTALL

When your project lands on Streamlit Cloud, the server starts with a clean Python install. It has no idea you used scikit-learn or pandas. `requirements.txt` is the shopping list. List each library on its own line, and pin the same version you used when training so the model unpickles cleanly.

REQUIREMENTS.TXT

```
streamlit==1.39.0  
scikit-learn==1.5.2  
pandas==2.2.3  
joblib==1.4.2  
numpy==1.26.4
```

Check your training environment with `pip freeze` and copy the exact versions for any library your model depends on.



• STEP 5

PUSH TO GITHUB

Streamlit Cloud reads your code from a GitHub repository. Create a new empty repo on github.com, then from inside your project folder, run these commands. Replace the URL with the one GitHub gives you.

```
$ git init
$ git add app.py model.pkl requirements.txt
$ git commit -m "first version of my streamlit app"
$ git branch -M main
$ git remote add origin https://github.com/yourname/my-app.git
$ git push -u origin main
```

TERMINAL

! Notice `model.pkl` is in the `git add` line. The cloud cannot magically know about your model. It has to live in the repo.



• STEP 6

DEPLOY ON STREAMLIT COMMUNITY CLOUD

This is the easy part. It is free, no credit card. Four clicks and you have a public URL.

01

Visit the site

Go to `share.streamlit.io`.

02

Sign in

Use your GitHub account so it can see your repos.

03

New app

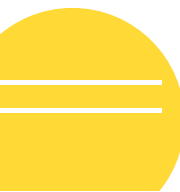
Click **New app**. Pick the repo, branch `main`, and the file `app.py`.

04

Deploy

Click **Deploy**. In ~2 minutes you get a live URL.

Your URL will look like `yourname-my-app-app-abc123.streamlit.app`. Share it anywhere.



• PITFALL 1 OF 3

VERSION MISMATCH

The most common deployment failure looks like a frightening error about NumPy or sklearn the moment the app tries to load `model.pkl`. The cause is almost always the same: you trained the model with one version of scikit-learn, and the cloud installed a different one. Pickle files are sensitive to library versions.

The fix

Run `pip freeze` in the same environment you used for training. Copy the exact versions of `scikit-learn`, `numpy`, and any other library your model touches into `requirements.txt`. Pin them with `=`, not `>=`.

THE MODEL FILE IS MISSING

You push your code, deploy, and get `FileNotFoundError: model.pkl`. The model file never made it to GitHub. Either you forgot `git add model.pkl`, or a `.gitignore` rule is excluding `.pkl` files (some Python templates do this by default).

How to check

From your project folder run:

```
$ git ls-files | grep pkl  
model.pkl
```

If the file is not listed, it is not tracked. Add it, commit, push again.



• PITFALL 3 OF 3

MODEL FILE IS TOO BIG

GitHub rejects any single file larger than 100MB. If your `model.pkl` is 200MB, the push will fail with a clear error. Random Forests with hundreds of trees, or models trained on thousands of features, get heavy fast.

Option A: shrink the model

Use fewer trees, fewer features, a simpler algorithm. Often the smaller model performs almost the same and is the cleaner answer for a teaching project.

Option B: Git LFS

Git Large File Storage handles binaries above 100MB. More setup, but useful when the big model is genuinely needed.



• A STARTING POINT

A GENERIC APP.PY YOU CAN REUSE

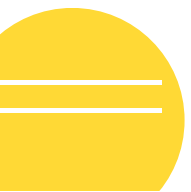
The same `app.py` works for almost any simple sklearn model. You only edit two things: the `FEATURES` list (one entry per input column your model expects), and the optional `CLASS_LABELS` dictionary if you have a classifier and want friendly names.

```
APP.PY (EXCERPT)

# 1. Edit this list to match your model's input features
FEATURES = [
    {"name": "sepal_length", "label": "Sepal length (cm)", "min": 0.0, "max": 10.0, "default": 5.0},
    {"name": "sepal_width", "label": "Sepal width (cm)", "min": 0.0, "max": 10.0, "default": 3.0},
    # add one entry per feature your model was trained on
]

# 2. Optional: friendly labels for classifier outputs
CLASS_LABELS = {0: "setosa", 1: "versicolor", 2: "virginica"}
```

Find the full file at `tasks/streamlit-deployment/app.py`. Drop your `model.pkl` in the same folder, edit those two variables, and you have a working app.



SIX STEPS, FROM NOTEBOOK TO URL

01

Save the model

```
joblib.dump(model, "model.pkl")
```

02

Write app.py

Load the model, collect inputs, predict, show.

03

Run locally

```
streamlit run app.py
```

04

Add requirements.txt

Pin every library version your model needs.

05

Push to GitHub

Including `model.pkl`. Always.

06

Deploy on the cloud

share.streamlit.io, four clicks, public URL.



ANY QUESTIONS?

Now go and put a model on the internet. The world is waiting to click
your button.

